Chapter 3. Looking Inside Large Language Models

The Inputs and Outputs of a Trained Transformer LLM

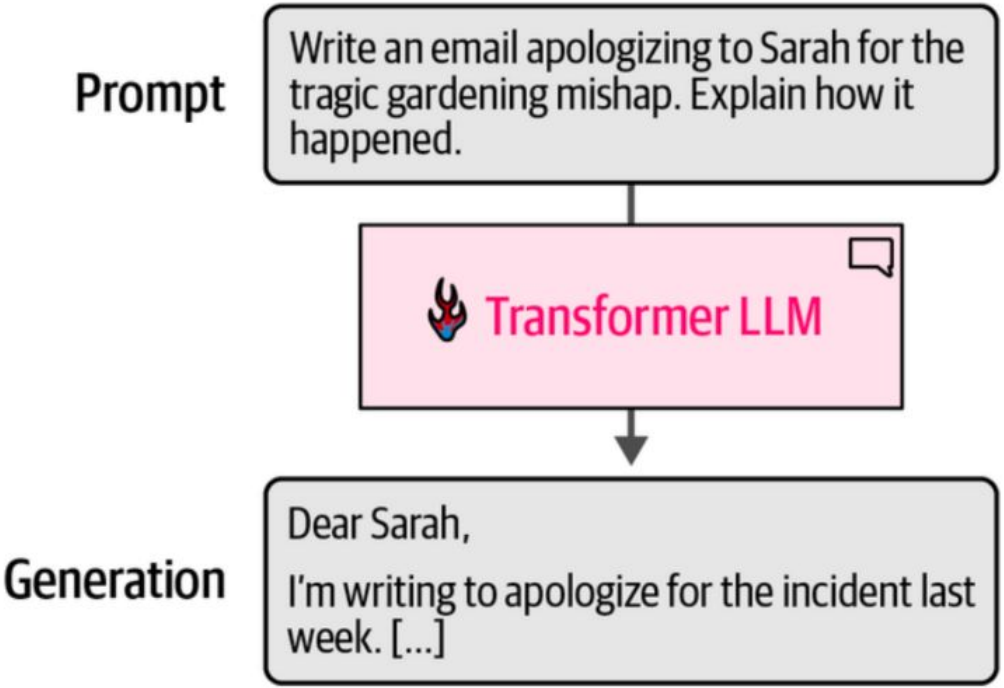


Figure 3-1. At a high level of abstraction, Transformer LLMs take a text prompt and output generated text.

The model does not generate the text all in one operation; it actually generates one token at a time.

Figure 3-2 shows four steps of token generation in response to the input prompt.

Each token generation step is one forward pass through the model (that's machine-learning speak for the inputs going into the neural network and flowing through the computations it needs to produce an output on the other end of the computation graph).

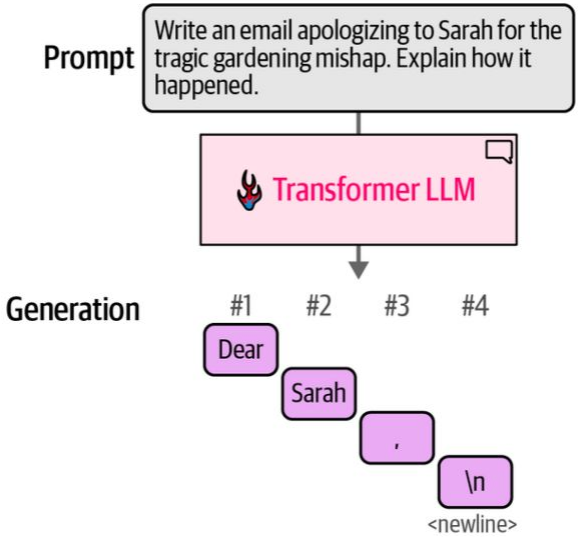


Figure 3-2. Transformer LLMs generate one token at a time, not the entire text at once.

Appending the Output again as Input

After each token generation, we tweak the input prompt for the next generation step by appending the output token to the end of the input prompt. We can see this in Figure 3-3.

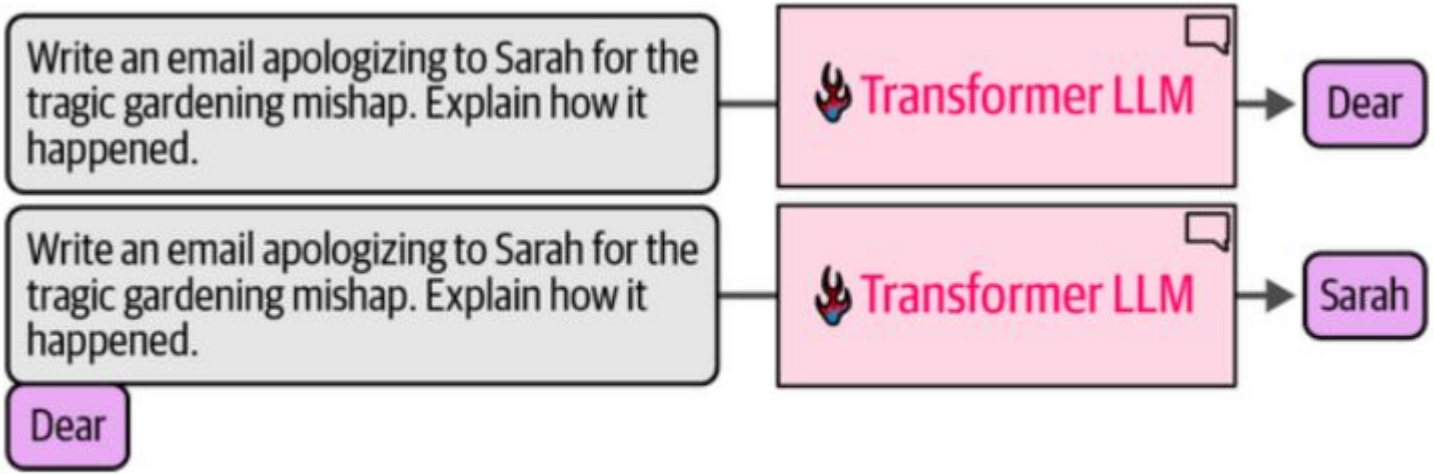


Figure 3-3. An output token is appended to the prompt, then this new text is presented to the model again for another forward pass to generate the next token.

Autoregressive vs. Representation models

There’s a specific word used in machine learning to describe models that consume their earlier predictions to make later predictions (e.g., the model’s first generated token is used to generate the second token). They’re called autoregressive models.

That is why you’ll hear text generation LLMs being called autoregressive models.

This is often used to differentiate text generation models from text representation models like BERT, which are not autoregressive.

The Components of the Forward Pass

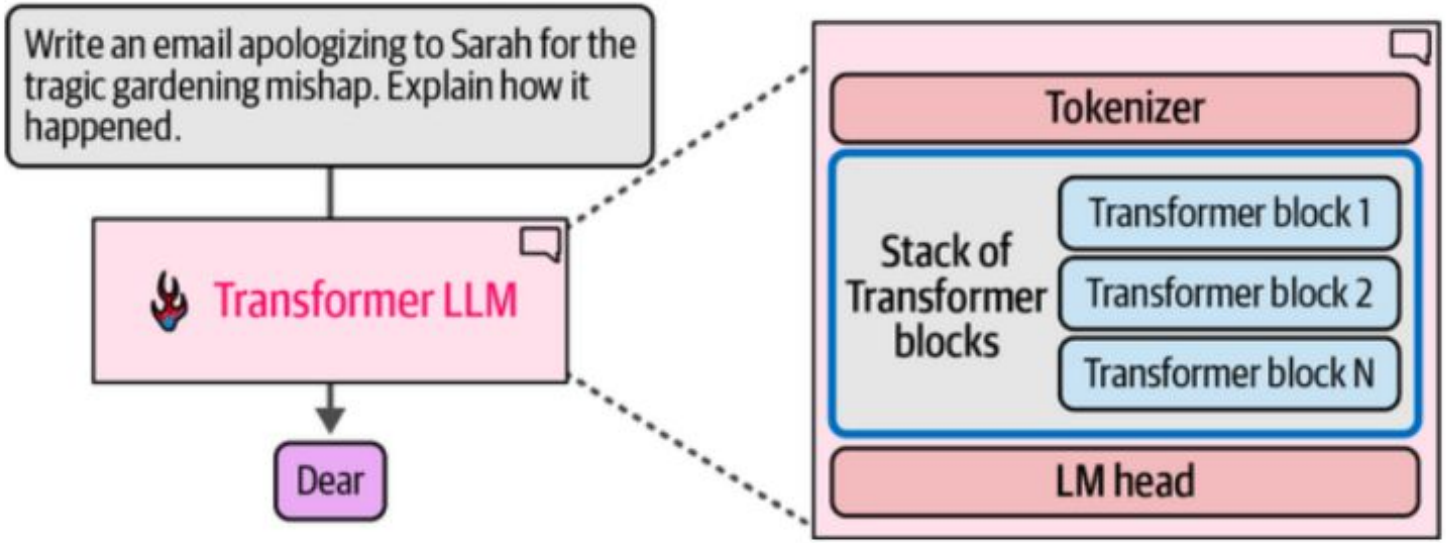


Figure 3-4. A Transformer LLM is made up of a tokenizer, a stack of Transformer blocks, and a language modeling head.

The tokenizer is followed by the neural network: a stack of Transformer blocks that do all of the processing. That stack is then followed by the LM head, which translates the output of the stack into probability scores for what the most likely next token is.

Flow of Control During Token Generation

For each generated token, the process flows once through each of the Transformer blocks in the stack in order, then to the LM head, which finally outputs the probability distribution for the next token, seen in Figure 3-6.

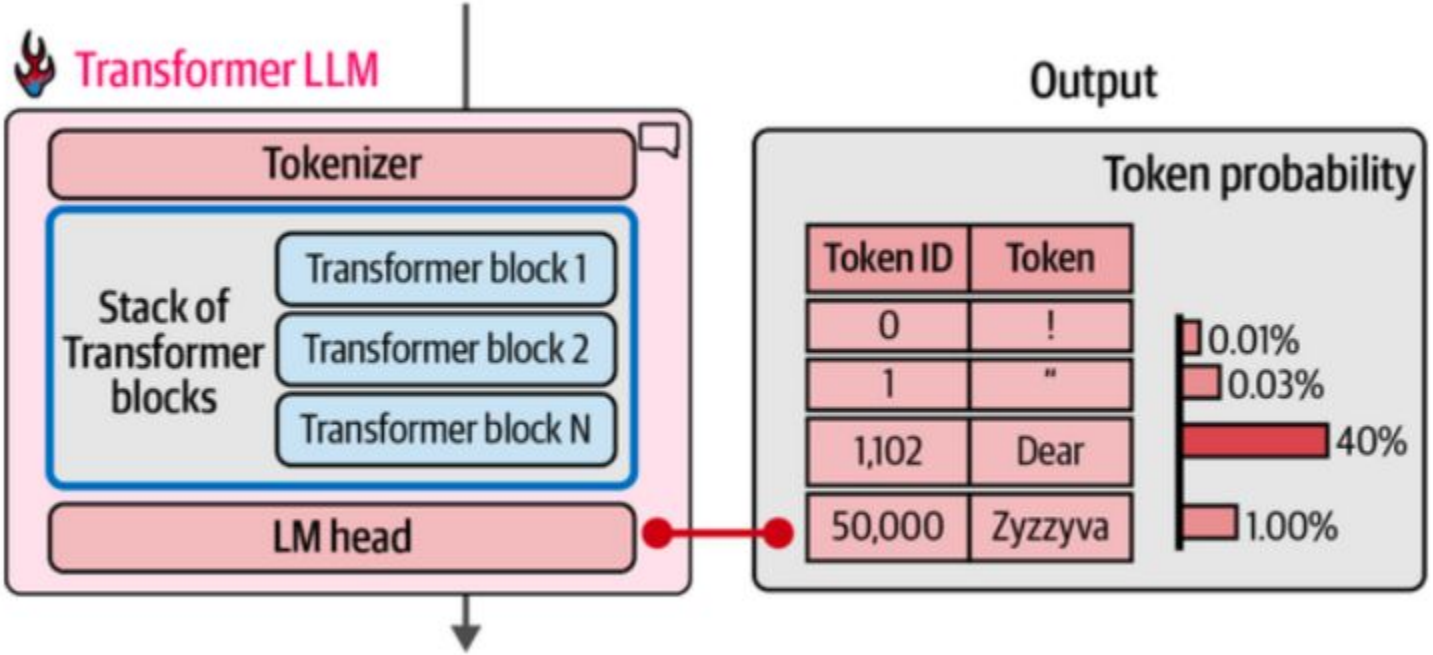


Figure 3-6. At the end of the forward pass, the model predicts a probability score for each token in the vocabulary.

Choosing a Single Token from the Probability Distribution (Sampling/Decoding)

At the end of processing, the output of the model is a probability score for each token in the vocabulary, as we saw previously in Figure 3-6. The method of choosing a single token from the probability distribution is called the decoding strategy.

The easiest decoding strategy would be to always pick the token with the highest probability score. In practice, this doesn't tend to lead to the best outputs for most use cases. A better approach is to add some randomness and sometimes choose the second or third highest probability token. The idea here is to basically sample from the probability distribution based on the probability score, as the statisticians would say.

Example - Token Probabilities

Let's look more closely at the code that demonstrates this process. In this code block, we pass the input tokens through the model, and then `lm_head`:

```
prompt = "The capital of France is"

# Tokenize the input prompt
input_ids = tokenizer(prompt, return_tensors="pt").input_ids

# Tokenize the input prompt
input_ids = input_ids.to("cuda")

# Get the output of the model before the lm_head
model_output = model.model(input_ids)

# Get the output of the lm_head
lm_head_output = model.lm_head(model_output[0])
```

Example - Token Probabilities

Now, `lm_head_output` is of the shape [1, 6, 32064]. We can access the token probability scores for the last generated token using `lm_head_output[0,-1]`, which uses the index 0 across the batch dimension; the index –1 gets us the last token in the sequence. This is now a list of probability scores for all 32,064 tokens. We can get the top scoring token ID, and then decode it to arrive at the text of the generated output token:

```
token_id = lm_head_output[0,-1].argmax(-1)
tokenizer.decode(token_id)
```

In this case this turns out to be:

```
Paris
```

Parallel Token Processing and Context Size

In text generation, we get a first glance at this when looking at how each token is processed. We know from the previous chapter that the tokenizer will break down the text into tokens. Each of these input tokens then flows through its own computation path (that's a good first intuition, at least). We can see these individual processing tracks or streams in Figure 3-8.

Current Transformer models have a limit for how many tokens they can process at once. That limit is called the model's context length. A model with 4K context length can only process 4K tokens and would only have 4K of these streams.

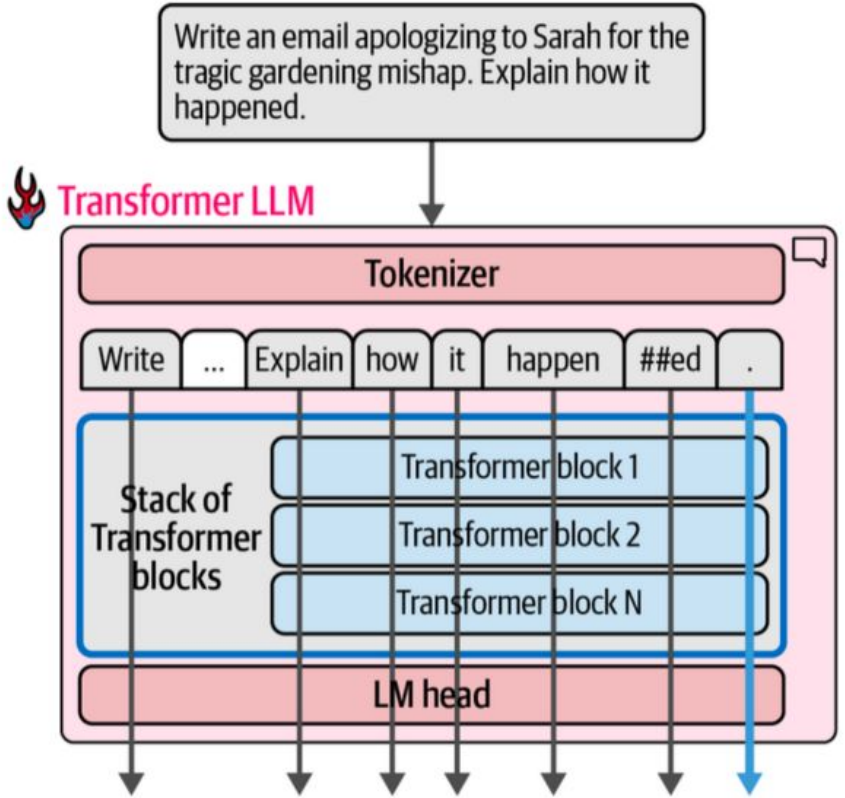


Figure 3-8. Each token is processed through its own stream of computation (with some interaction between them in attention steps, as we'll later see).

Speeding Up Generation by Caching Keys and Values

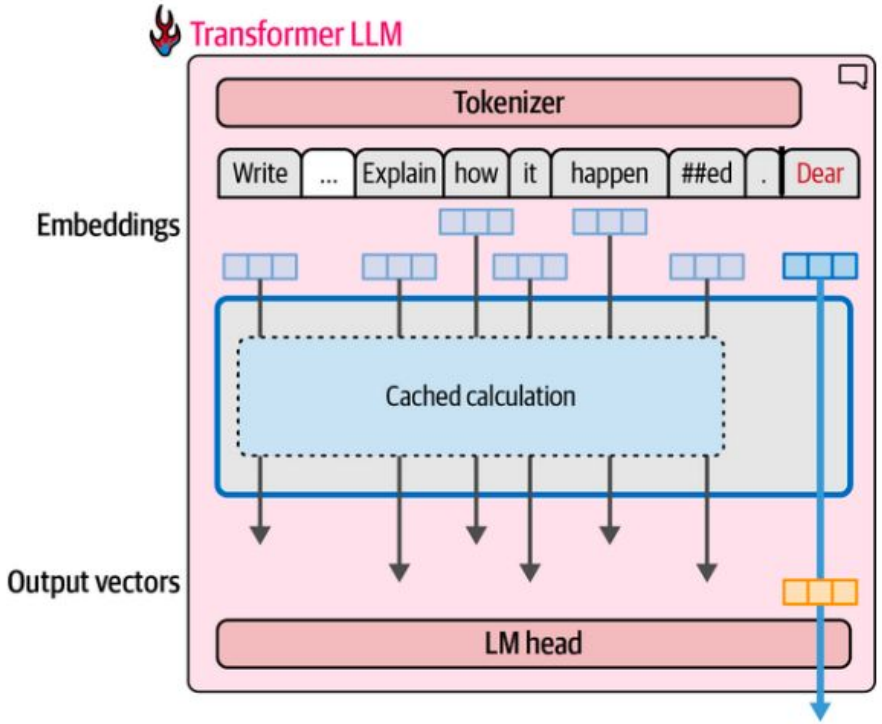


Figure 3-10. When generating text, it's important to cache the computation results of previous tokens instead of repeating the same calculation over and over again.

Inside the Transformer Block

As Figure 3-11 shows, Transformer LLMs are composed of a series Transformer blocks (often in the range of six in the original Transformer paper, to over a hundred in many large LLMs). Each block processes its inputs, then passes the results of its processing to the next block.

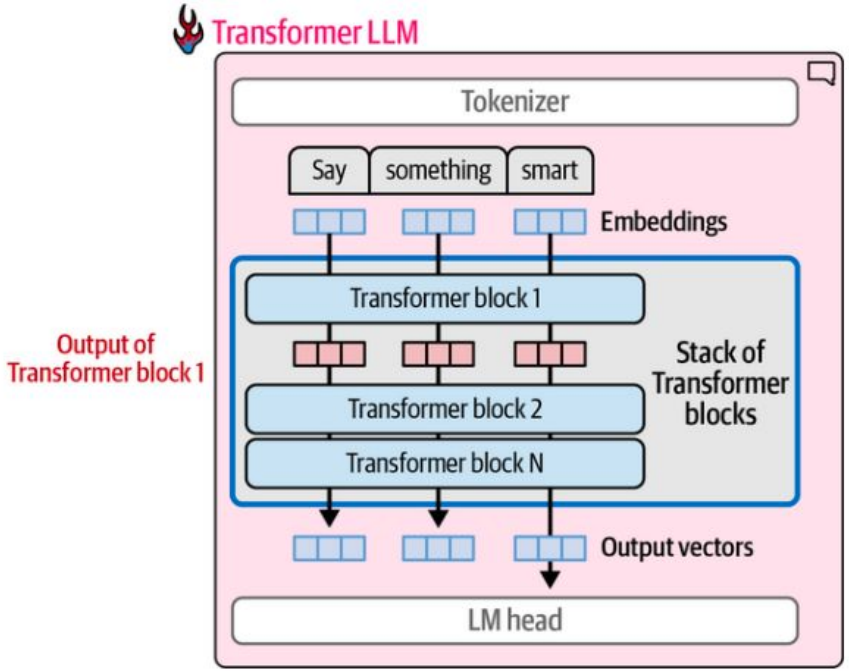


Figure 3-11. The bulk of the Transformer LLM processing happens inside a series of Transformer blocks, each handing the result of its processing as input to the subsequent block.

Each Transformer Block

A Transformer block (Figure 3-12) is made up of two successive components:

1. The attention layer is mainly concerned with incorporating relevant information from other input tokens and positions
2. The feedforward layer houses the majority of the model's processing capacity

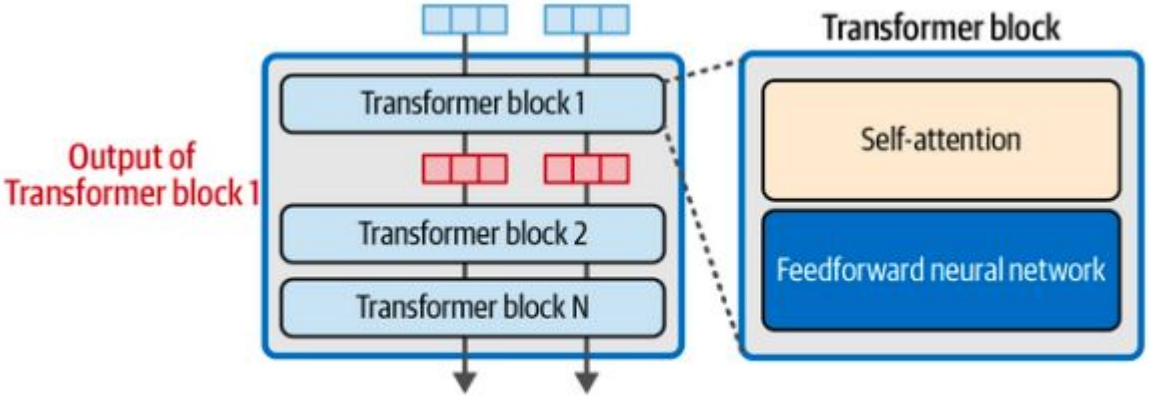


Figure 3-12. A Transformer block is made up of a self-attention layer and a feedforward neural network.

Feed Forward Neural Network

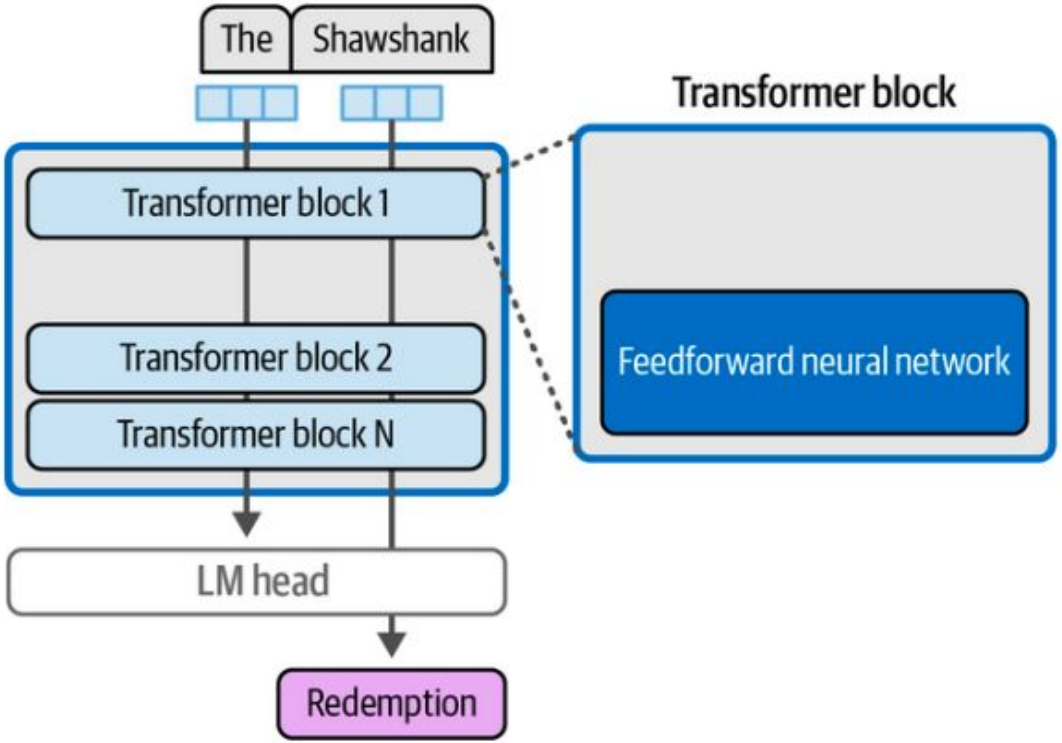


Figure 3-13. The feedforward neural network component of a Transformer block likely does the majority of the model's memorization and interpolation.

Attention Mechanism

- The Transformer architecture, introduced by Vaswani et al. (2017), marked a significant shift in natural language processing by replacing recurrent and convolutional structures with self-attention mechanisms.
- The Transformer architecture, proposed by Vaswani et al., introduces a novel approach that relies exclusively on attention mechanisms, eliminating the need for recurrence and convolutions Entirely.
- Attention is a mechanism that helps the model incorporate context as it'sprocessing a specific token. Think of the following prompt:

  "The dog chased the squirrel because **it**"

- For the model to predict what comes after "it," it needs to know what "it"refers to. Does it refer to the dog or the squirrel? In a trained Transformer LLM, the attention mechanism makes that determination. Attention adds information from the context into the representation of the "it" token. We can see a simple version of that in Figure 3-14.

Figure 3-15. It shows multiple token positions going into the attention layer; the final one is the one being currently processed (the pink arrow). The attention mechanism operates on the input vector at that position. It incorporates relevant information from the context into the vector it produces as the output for that position.

The attention layer helps the model understand each word by looking at the whole sentence and picking up useful clues from other words.

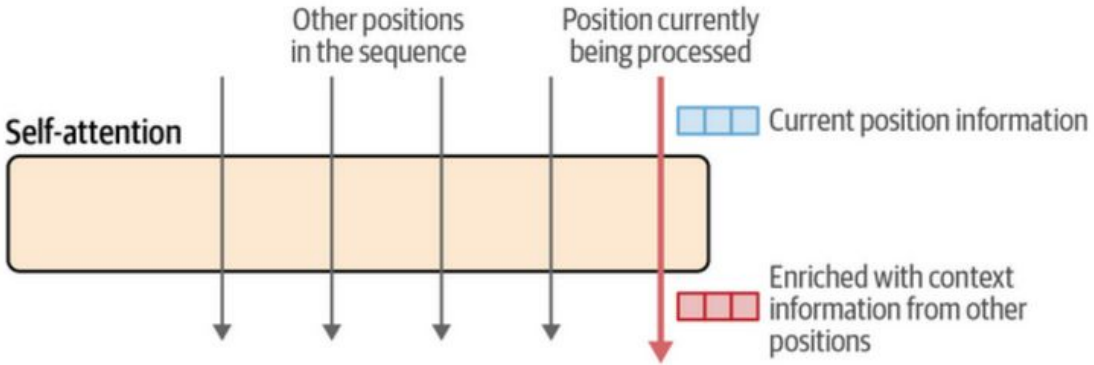


Figure 3-15. A simplified framing of attention: an input sequence and a current position being processed. As we're mainly concerned with this position, the figure shows an input vector and an output vector that incorporates information from the previous elements in the sequence according to the attention mechanism.

Two main steps are involved in the attention mechanism:

- A way to score how relevant each of the previous input tokens are to the current token being processed (in the pink arrow).
- Using those scores, we combine the information from the various positions into a single output vector.

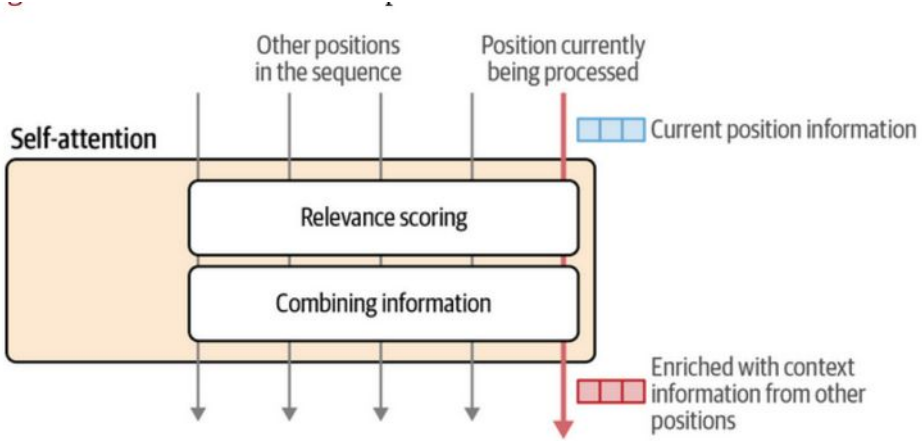


Figure 3-16. Attention is made up of two major steps: relevance scoring for each position, then a step where we combine the information based on those scores.

How attention is calculated

The training process produces three projection matrices that produce the components that interact in this calculation:

- A query projection matrix (Q)
- A key projection matrix (K)
- A value projection matrix (V)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V,$$

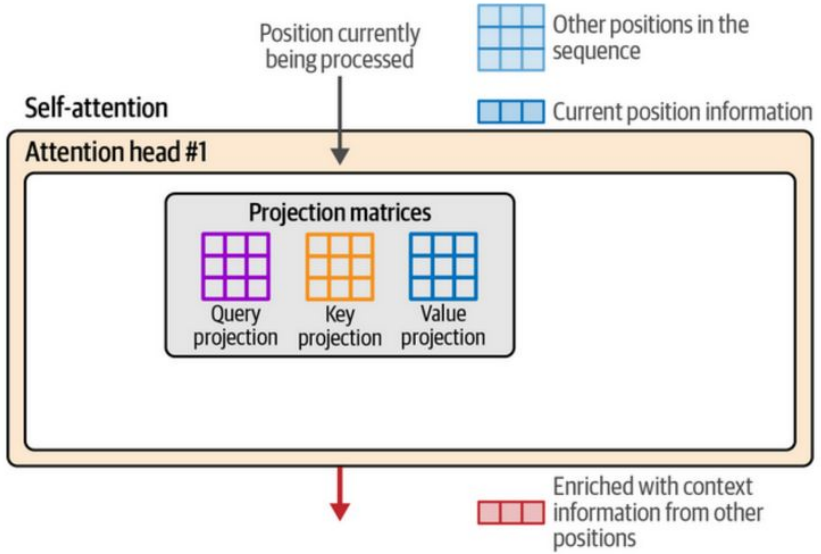


Figure 3-18. Before starting the self-attention calculation, we have the inputs to the layer and projection matrices for queries, keys, and values.

Relation between Q, K, V

Each token generates a **query** vector that represents what it is “looking for” in other tokens. This vector is used to compute similarity with the **key** vectors of all tokens in the sequence.

Let’s say Q_sat = [1, 0]

This means "sat" is looking for words that are strong in **feature 1**.

Now we compare Q_sat to the **K vectors of all other words**:

Word	K vector	Dot product with Q_sat
The	[1, 0]	1×1 + 0×0 = 1 ✅
cat	[0, 1]	1×0 + 0×1 = 0 ❌
mat	[1, 1]	1×1 + 0×1 = 1 ✅

So "sat" pays attention to "The" and "mat", but not "cat".

Then it uses the **V vectors** of those words to build a new meaning for "sat".

Softmax Function

The **softmax function** transforms a vector of raw scores into a probability distribution. It is defined as:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

This ensures that all output values lie between 0 and 1 and sum to 1. In the Transformer architecture, softmax is used to compute attention weights from similarity scores and to generate token probabilities in the final output layer.

Example: Consider the vector of scores:

$$\mathbf{z} = [2.0,\ 1.0,\ 0.1]$$

First, exponentiate each element:

$$e^{2.0} \approx 7.39, \quad e^{1.0} \approx 2.72, \quad e^{0.1} \approx 1.11$$

Next, compute the sum:

$$\sum e^{z_j} = 7.39 + 2.72 + 1.11 = 11.22$$

Now, apply softmax:

$$\text{softmax}(2.0) = \frac{7.39}{11.22} \approx 0.659, \quad \text{softmax}(1.0) = \frac{2.72}{11.22} \approx 0.242, \quad \text{softmax}(0.1) = \frac{1.11}{11.22} \approx 0.099$$

Thus, the output is:

$$[0.659,\ 0.242,\ 0.099]$$

This vector represents a probability distribution over the input scores.

Example - Calculating the Attention Scores

In the attention mechanism, each token computes its relevance to other tokens using the dot product between its query vector and the key vectors of all tokens. These raw scores are then scaled and normalized to produce attention weights.

Let dk be the dimension of the key vectors. To prevent extremely large dot products that could destabilize training, we scale the raw scores by $\sqrt{dk}$. The scaled scores are then passed through a softmax function to obtain the final attention weights.

In this example, we compute the attention weights for the token “crossed” in the sentence “The chicken crossed the road.” The goal is to determine how much “crossed” attends to each token in the sequence

Assume each token is projected into a 4-dimensional space, so $d_k = 4$ and the scaling factor is $\sqrt{4} = 2$. Let the query vector for "crossed" be:

$$Q_{\text{crossed}} = [1, 2, 1, 0]$$

And the key vectors for the tokens in the sentence "The chicken crossed the road" are:

$$\begin{aligned} K_{\text{The}} &= [1, 1, 0, 0] \\ K_{\text{Chicken}} &= [2, 1, 1, 0] \\ K_{\text{Crossed}} &= [1, 2, 1, 0] \\ K_{\text{The}} &= [1, 1, 0, 0] \\ K_{\text{Road}} &= [1, 1, 1, 0] \end{aligned}$$

We compute the dot product between Q_{crossed} and each K_i:

$$\begin{aligned} Q \cdot K_{\text{The}} &= 1 \times 1 + 2 \times 1 + 1 \times 0 + 0 \times 0 = 3 \\ Q \cdot K_{\text{Chicken}} &= 1 \times 2 + 2 \times 1 + 1 \times 1 + 0 \times 0 = 5 \\ Q \cdot K_{\text{Crossed}} &= 1 \times 1 + 2 \times 2 + 1 \times 1 + 0 \times 0 = 6 \\ Q \cdot K_{\text{Road}} &= 1 \times 1 + 2 \times 1 + 1 \times 1 + 0 \times 0 = 4 \end{aligned}$$

Then we scale each score by dividing by $\sqrt{d_k} = 2$:

$$\text{Scaled}_{\text{The}} = \frac{3}{2} = 1.5$$

$$\text{Scaled}_{\text{Chicken}} = \frac{5}{2} = 2.5$$

$$\text{Scaled}_{\text{Crossed}} = \frac{6}{2} = 3.0$$

$$\text{Scaled}_{\text{Road}} = \frac{4}{2} = 2.0$$

Apply softmax to these scaled scores:

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Let's compute:

$$
\begin{aligned}
e^{1.5} &\approx 4.48 \\
e^{2.5} &\approx 12.18 \\
e^{3.0} &\approx 20.09 \\
e^{2.0} &\approx 7.39 \\
\text{Sum} &= 4.48 + 12.18 + 20.09 + 7.39 = 44.14
\end{aligned}
$$

Final attention weights:

$$
\begin{aligned}
\alpha_{\text{The}} &= \frac{4.48}{44.14} \approx 0.10 \\
\alpha_{\text{Chicken}} &= \frac{12.18}{44.14} \approx 0.28 \\
\alpha_{\text{Crossed}} &= \frac{20.09}{44.14} \approx 0.46 \\
\alpha_{\text{Road}} &= \frac{7.39}{44.14} \approx 0.17
\end{aligned}
$$

These weights determine how much "crossed" attends to each token. The highest attention is to itself, followed by "chicken" and "road."

Positional Embeddings (RoPE)

- Positional embeddings have been a key component since the original Transformer. They enable the model to keep track of the order of tokens/words in a sequence/sentence, which is an indispensable source of information in language.
- From the many positional encoding schemes proposed in the past years, rotary positional embeddings (or “RoPE,” introduced in “RoFormer: Enhanced Transformer with rotary position embedding”) is especially important to point out.